

A SQL-Based Project

FRAUD DETECTION SYSTEM

SQL

Transaction Monitoring

Developed By :
Shweta Mourya

ABSTRACT

This project presents a SQL-based Fraud Detection System designed to identify suspicious financial transactions through systematic rule-based analysis. The system evaluates transaction patterns using multiple detection techniques, including high-amount checks, velocity rules, blacklist matching, and anomaly identification. By processing historical and real-time transaction data, the model highlights potentially fraudulent activity and stores the results in an alert table for monitoring and investigation. This SQL-driven approach provides a simple, efficient, and scalable framework for fraud detection, enabling organizations to reduce financial risks and ensure secure transaction processing.

ER DIAGRAM



accounts

PK account_id INT
customer_name FSS
created_at TIMESTAMP



alerts

PK alert_id INT
PK transaction_id INT
rule_triggered VARCHAR
created_at TIMESTAMP



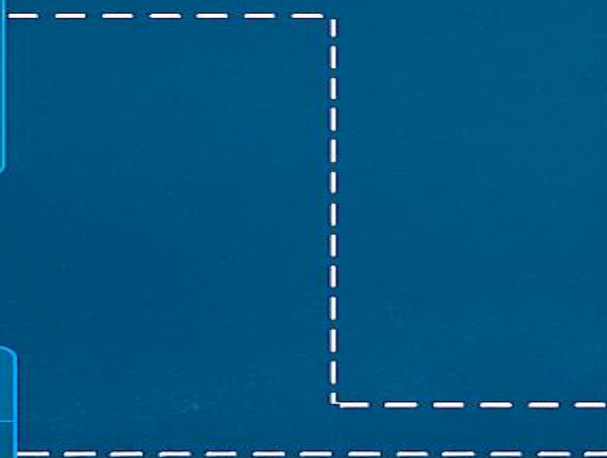
transactions

FK blacklist_id INT
PK type VARCHAR(20)
PK value VARCHAR(100)
reason TEXT



blacklists

PK transaction_id INT
PK account_id INT
amount DECIMAL(2,2)
txn_time TIMESTAMPT
device_id VARCHAR(50)
geo_idrass VARCHAR(48)
geo_lat DOUBLE



Structure of Table

➤ ACCOUNTS TABLE

```
describe accounts;
```

	Field	Type	Null	Key	Default	Extra
▶	account_id	int(11)	NO	PRI	HULL	auto_increment
	customer_name	varchar(150)	YES		HULL	
	created_at	timestamp	NO		current_timestamp()	

- Stores information about each user/account in the system. Fraud checks are performed *per account*, so this table helps link all transactions back to the user.

Structure of Table

➤ TRANSACTION TABLE

```
describe transactions;
```

	Field	Type	Null	Key	Default	Extra
▶	transaction_id	int(11)	NO	PRI	NULL	auto_increment
	account_id	int(11)	NO	MUL	NULL	
	amount	decimal(12,2)	YES		NULL	
	txn_time	timestamp	NO		current_timestamp()	on update current_timestamp()
	device_id	varchar(50)	YES		NULL	
	ip_address	varchar(50)	YES		NULL	
	geo_lat	double	YES		NULL	
	geo_lon	double	YES		NULL	

- Stores every transaction done by an account, including amount, time, device, and IP. This is the core table used for fraud detection. Rules like high amount, velocity, or suspicious IP are applied here.

Structure of Table

➤ BLACKLISTS TABLE

```
describe blacklists;
```

	Field	Type	Null	Key	Default	Extra
►	blacklist_id	int(11)	NO	PRI	NULL	auto_increment
	type	varchar(20)	YES		NULL	
	value	varchar(100)	YES		NULL	
	reason	text	YES		NULL	

- Stores blocked/suspicious items such as IP addresses, accounts, or devices. If a transaction matches an entry in the blacklist, it is immediately flagged as fraud.

Structure of Table

➤ ALERT TABLE

```
describe alerts;
```

	Field	Type	Null	Key	Default	Extra
►	alert_id	int(11)	NO	PRI	NULL	auto_increment
	transaction_id	int(11)	YES	MUL	NULL	
	rule_triggered	varchar(50)	YES		NULL	
	created_at	timestamp	NO		current_timestamp()	

- Stores all fraud warnings generated by your rules (e.g., HIGH_AMOUNT, BLACKLIST_MATCH). This table acts as the fraud monitoring dashboard, helping investigators review suspicious transactions.

Content of Table

```
SELECT * FROM accounts;
```

	account_id	customer_name	created_at
▶	1	Neha Sharma	2025-12-08 09:44:40
	2	Vikram Joshi	2025-12-08 09:44:40
	3	Pooja Mehta	2025-12-08 09:44:40
	4	Suresh Desai	2025-12-08 09:44:40
	5	Kiran Rao	2025-12-08 09:44:40
	6	Manish Verma	2025-12-08 09:44:40
	7	Deepa Iyer	2025-12-08 09:44:40
	8	Samir Purohit	2025-12-08 09:44:40
	9	Anita Kapoor	2025-12-08 09:44:40
	10	Prakash Yadav	2025-12-08 09:44:40
	11	Harsha Kulkarni	2025-12-08 09:44:40
•	NULL	NULL	NULL


```
SELECT * FROM transactions;
```

[illegible]

Content of Table

```
SELECT * FROM blacklists;
```

	blacklist_id	type	value	reason
▶	1	ip	9.9.9.9	Known fraud IP used in multiple chargebacks
	2	device	devX123	Suspicious device flagged earlier
	3	account	3	Account linked to repeated fraudulent activity
●	NULL	NULL	NULL	NULL

Content of Table

```
SELECT * FROM alerts;
```

	alert_id	transaction_id	rule_triggered	created_at
▶	1	1	HIGH_AMOUNT	2025-12-08 09:45:09
	2	2	VELOCITY	2025-12-08 09:45:09
	3	3	BLACKLIST_IP	2025-12-08 09:45:09
	4	1	NEW_DEVICE	2025-12-08 09:45:09
	5	2	AMOUNT_OUTLIER	2025-12-08 09:45:09
⊛	NULL	NULL	NULL	NULL

JOINS

A join is used to combine rows from two or more tables based on a related column, allowing you to retrieve data from multiple tables in a single query.



- ❑ Display all transactions along with the customer name who performed them.

```
SELECT
    a.customer_name,
    t.transaction_id,
    t.amount,
    t.txn_time
FROM accounts a
JOIN transactions t
ON a.account_id = t.account_id;
```

➤ OUTPUT

	customer_name	transaction_id	amount	txn_time
▶	Neha Sharma	1	12000.00	2025-12-07 09:44:56
	Neha Sharma	2	200.00	2025-12-08 09:34:56
	Neha Sharma	3	250.00	2025-12-08 09:35:56
	Neha Sharma	4	300.00	2025-12-08 09:36:56
	Neha Sharma	5	350.00	2025-12-08 09:37:56
	Neha Sharma	6	400.00	2025-12-08 09:38:56

Conclusion:-This JOIN connects the accounts table with transactions using account_id, showing which customer performed each transaction.

- ❑ Show all transactions that were flagged as alerts along with the rule triggered.

```
SELECT
    t.transaction_id,
    t.amount,
    a.rule_triggered,
    a.created_at AS alert_time
FROM transactions t
JOIN alerts a
ON t.transaction_id = a.transaction_id;
```

➤ OUTPUT

	transaction_id	amount	rule_triggered	alert_time
▶	1	12000.00	HIGH_AMOUNT	2025-12-08 09:45:09
	2	200.00	VELOCITY	2025-12-08 09:45:09
	3	250.00	BLACKLIST_IP	2025-12-08 09:45:09
	1	12000.00	NEW_DEVICE	2025-12-08 09:45:09
	2	200.00	AMOUNT_OUTLIER	2025-12-08 09:45:09

Conclusion:- This JOIN shows which transactions generated fraud alerts, and what rule flagged them.

- ❑ List transactions that match any blacklisted IP along with the blacklist reason.

```
SELECT
    t.transaction_id,
    t.account_id,
    t.ip_address,
    b.reason
FROM transactions t
JOIN blacklists b
ON b.type = 'ip'
AND b.value = t.ip_address;
```

➤ OUTPUT

	transaction_id	account_id	ip_address	reason
▶	1	1	1.1.1.1	Suspicious IP detected in past fraud cases
	2	1	1.1.1.1	Suspicious IP detected in past fraud cases
	3	1	1.1.1.1	Suspicious IP detected in past fraud cases
	4	1	1.1.1.1	Suspicious IP detected in past fraud cases
	5	1	1.1.1.1	Suspicious IP detected in past fraud cases
	6	1	1.1.1.1	Suspicious IP detected in past fraud cases

This JOIN checks IP addresses in transactions and matches them with blacklisted IPs.
If a match occurs → transaction is suspicious.

SUBQUERY

A subquery is a query inside another query.
It is executed first, and the result is used by the outer (main) query.



❑ Find accounts that have at least one alert.

```
Select account_id  
From transactions  
where transaction_id IN(  
Select transaction_id from alerts  
);
```

➤ OUTPUT

	account_id
▶	1
	1
	1

The inner query selects all alerted transactions, the outer query finds which accounts they belong to.

- ❑ Show transactions greater than the average amount.

```
select transaction_id,amount  
From transactions  
where amount>(select avg(amount) from transactions);
```

➤ OUTPUT

	transaction_id	amount
▶	1	12000.00
●	NULL	NULL

The subquery calculates the **average amount**, and the outer query returns transactions above that value.

❑ Find all transactions whose amount is greater than the minimum transaction amount.

```
Select transaction_id, amount  
from transactions  
where amount > (select min(amount) from transactions);
```

➤ OUTPUT

	transaction_id	amount
▶	1	12000.00
	3	250.00
	4	300.00
	5	350.00
	6	400.00
*	NULL	NULL

The subquery finds the smallest transaction amount.

The outer query returns all transactions greater than that value.

WINDOW FUNCTION

A window function performs calculations across a set of rows related to the current row without grouping or losing detail.



❑ Running total of transactions per account.

```
SELECT
    account_id,
    transaction_id,
    amount,
    COUNT(*) OVER (PARTITION BY account_id ORDER BY txn_time)
        AS running_count
FROM transactions;
```

➤ OUTPUT

	account_id	transaction_id	amount	running_count
▶	1	1	12000.00	1
	1	2	200.00	2
	1	3	250.00	3
	1	4	300.00	4
	1	5	350.00	5
	1	6	400.00	6

Shows how many transactions the account has made **up to each row** (velocity pattern).

- ❑ Rank accounts based on total transaction amount.

```
SELECT
    account_id,
    SUM(amount) AS total_amount,
    RANK() OVER (ORDER BY SUM(amount) DESC) AS rank_no
FROM transactions
GROUP BY account_id;
```

➤ OUTPUT

	account_id	total_amount	rank_no
▶	1	13500.00	1

Ranks accounts by spending — used in fraud to detect abnormally high activity.

- ❑ Show each transaction with the average amount per account.

```
SELECT
    transaction_id,
    account_id,
    amount,
    AVG(amount) OVER (PARTITION BY account_id) AS avg_amount
FROM transactions;
```

➤ OUTPUT

	transaction_id	account_id	amount	avg_amount
▶	1	1	12000.00	2250.000000
	2	1	200.00	2250.000000
	3	1	250.00	2250.000000
	4	1	300.00	2250.000000
	5	1	350.00	2250.000000
	6	1	400.00	2250.000000

Window function shows average amount for each account while still displaying each transaction.

FINAL CONCLUSION

- The SQL-based fraud detection system successfully identified suspicious transactions using rule-based checks such as high-amount flags, velocity patterns, and blacklist matching.
- Transaction analysis revealed unusual spending behaviors and abnormal frequency patterns that can indicate potential fraud.
- Blacklist integration enabled early detection of fraudulent devices, accounts, and IP addresses, strengthening overall security.
- Time-based insights helped highlight unusual transaction spikes during non-business hours, supporting risk assessment and monitoring.
- Window functions and subqueries improved the accuracy of fraud scoring by analyzing account-level trends and historical patterns.
- Alerts generated from the detection rules provided actionable insights for timely investigation and fraud prevention.
- SQL joins enabled combining data from multiple tables, creating unified fraud reports that support security teams in decision-making.
- Overall, this project demonstrates how a structured SQL-driven approach can significantly enhance fraud monitoring, strengthen financial protection, and improve operational efficiency.

THANK YOU!!